

CS-768 (Learning with Graphs) Course Project Report

GraphTransLWG: Hybrid GNN–Transformer for Long-Range Graph Reasoning

Eshaan Pareek (22B3986) Tejas Sharma (22B0909)
Aditya Vema Reddy Kesari (22B3985) Matam Kushaal (23B1290)

1 Introduction

Graph Neural Networks (GNNs) are powerful tools for learning from structured data, but they suffer from a fundamental tension: stacking more layers to widen the receptive field leads to oversmoothing and oversquashing, while shallow networks fail to model long-range dependencies [12]. This project implements a **GraphTrans-style** architecture [1], a hybrid design that combines a local GNN backbone with a global Transformer.

Our implementation, GRAPHTRANSLWG, follows the paper’s central design pattern—[CLS]-token readout, reachability-masked global attention, and pre-norm residual blocks—while introducing several implementation-level choices of its own: a custom edge-aware GNN layer, transitive-closure-based attention masking precomputed once per batch, and separate MLP weights for the [CLS] token. We evaluate on the NCI biological benchmarks and OGB Code2, and evaluate a readout ablation for the [CLS] embedding.

2 Problem Setup

Given a graph $G = (V, E)$ with per-node feature matrix $\mathbf{X} \in \mathbb{R}^{|V| \times d_x}$ and graph-level label y , learn $f_\theta(G) \rightarrow \hat{y}$ for whole-graph classification or sequence prediction.

Message passing (local bias). A standard GNN layer updates node i as

$$\vec{h}_i^{(\ell+1)} = \phi \left(\vec{h}_i^{(\ell)}, \bigoplus_{j \in \mathcal{N}(i)} \psi \left(\vec{h}_i^{(\ell)}, \vec{h}_j^{(\ell)} \right) \right),$$

where $\bigoplus$ is a permutation-invariant aggregation operator. Stacking k such layers gives a receptive field of k hops, but deeper stacks collapse representations—a phenomenon called *oversmoothing*.

Global attention (long-range context). Given hidden matrix $H \in \mathbb{R}^{N \times d}$ for all N nodes, multi-head scaled dot-product attention computes

$$\mathbf{Q} = \mathbf{H} @ \mathbf{W}_Q^\top, \quad \mathbf{K} = \mathbf{H} @ \mathbf{W}_K^\top, \quad \mathbf{V} = \mathbf{H} @ \mathbf{W}_V^\top, \quad \text{Attn}(\mathbf{H}) = \text{softmax} \left(\frac{\mathbf{Q} @ \mathbf{K}^\top}{\sqrt{d_h}} + \mathbf{M} \right) @ \mathbf{V},$$

where M is an additive attention bias (mask). Without sequence positional encodings, attention is permutation-equivariant over node order; after a graph-level readout such as [CLS], this gives a permutation-invariant graph prediction. Every node can attend to every reachable node permitted by the mask in a single attention layer, bypassing part of the message-passing depth bottleneck.

3 Architecture

The `GraphTransModel` (in `models/full_model.py`) consists of:

1. **CLS token injection:** append a [CLS] embedding to the graph (or each graph in the batch), prepare the batch edge index and the CLS mask.
2. **Input embedding:** project raw node features to dimension d , and use a learnable [CLS] embedding for each CLS node in the batch.
3. **Transformer stack:** L transformer blocks, each containing a GNN sublayer, a multi-head attention sublayer, and a feedforward MLP sublayer with pre-norm residuals.
4. **Readout:** extract the [CLS] embedding and apply a linear output head (or output heads, in case of OGBG Code2)

3.1 Batch Packing (`models/batch_utils.py`)

Before any forward pass, `pack_graph_batch` adds a CLS node to each graph, and then concatenates all graphs in the batch into a single flat vertex tensor and precomputes three structures that are reused across all L transformer layers:

- **cls_mask:** a boolean vector marking the CLS positions. For a batch of graphs with $[n_1, n_2, \dots]$ nodes, the CLS of graph i sits at index $\sum_{j < i} (n_j + 1) + n_i$.
- **edge_index:** packed, offset-corrected edge list for the whole batch.
- **attn_factors:** the *transitive closure* of the adjacency matrix (plus CLS connections), computed via fixed-point iteration initialised from the identity matrix $\mathbf{I}$: $\mathbf{A}^* \leftarrow \text{clamp}(\mathbf{A}^* \cdot \mathbf{A}, 1)$ until convergence. Entry $(i, j) = 1$ iff node j is reachable from node i in zero or more hops (within the same graph); cross-graph entries remain 0. Its log is passed as an additive bias to `scaled_dot_product_attention`, driving unreachable pairs to $-\infty$.
 - If the parameter `attn_factors` is set (as in not `None`), the entries in $\mathbf{A}$ are weighed by the distance factor, to prioritize attention to nearby nodes. That is, the update step becomes $\mathbf{A}_n^* \leftarrow \text{clamp}(\max(f_n \mathbf{A}_{n-1}^* \cdot \mathbf{A}, \mathbf{A}_{n-1}^*), 1)$, and this is repeated till convergence, with $f_n = 0$ for out-of-range indices.

Precomputing once and caching as a `PackedGraphBatch` dataclass eliminates expensive per-layer reachability computation in Python.

3.2 Input Embedding (`models/full_model.py`)

For graph classification datasets (NCI1, NCI109, OGBG Molpcba), a single linear layer maps raw node features $\vec{x}_v \in \mathbb{R}^{d_x}$ to $\vec{h}_v^{(0)} \in \mathbb{R}^d$:

$$\vec{h}_v^{(0)} = \mathbf{W}_{\text{in}} \vec{x}_v.$$

For OGBG Code2 (Abstract Syntax Trees), node features consist of a discrete node type, a discrete attribute index, and an integer depth. The `ASTNodeEncoder` module therefore uses three separate embedding tables (type, attribute, depth) and sums their outputs:

$$\vec{h}_v^{(0)} = E_{\text{type}}[x_{v,0}] + E_{\text{attr}}[x_{v,1}] + E_{\text{depth}}[\min(\delta_v, d_{\text{max}})],$$

where δ_v is the node’s depth in the AST, clipped at $d_{\text{max}} = 20$. For Code2, the dataset transform also augments the AST edge set with inverse edges and next-token edges before graph packing.

The [CLS] token is represented by a learned linear map applied to a fixed ones vector—effectively a learnable bias vector of dimension d —with one CLS token appended *after* the node block of each graph in the packed batch.

3.3 Transformer Block (`models/transformer.py`)

Each of the L transformer blocks is a `TransformerLayer` that applies three sublayers with pre-norm residuals:

$$\begin{aligned}\mathbf{H} &\leftarrow \mathbf{H} + \text{Drop}(\text{GNN}(\text{LN}(\mathbf{H}))) \\ \mathbf{H} &\leftarrow \mathbf{H} + \text{Drop}(\text{Attn}(\text{LN}(\mathbf{H}))) \\ \mathbf{H} &\leftarrow \mathbf{H} + \text{Drop}(\text{MLP}(\text{LN}(\mathbf{H})))\end{aligned}$$

where $\mathbf{H} \in \mathbb{R}^{(N+B) \times d}$ holds all node and CLS embeddings for a batch of B graphs with N total nodes.

3.4 GNN Sublayer (`models/gnn.py`)

Each `GNNLayer` is a custom *edge-aware message-passing* step that operates **only on non-CLS nodes**; CLS positions receive zero output and rely on the residual to carry their values forward. The computation for node u is:

$$\begin{aligned}\vec{h}_u &= \vec{h}_u @ \mathbf{W}_1^\top && \text{(node transform)} \\ \vec{e}_{uv} &= [\vec{h}_u \parallel \vec{h}_v] @ \mathbf{W}_2^\top && \text{(edge embedding from pair)} \\ \vec{m}_u &= \sum_{v \in \mathcal{N}(u)} \vec{h}_v && \text{(sum of neighbour transforms)} \\ \vec{n}_u &= \sum_{v \in \mathcal{N}(u)} \vec{e}_{uv} && \text{(sum of edge embeddings)} \\ \vec{h}'_u &= \text{GELU}([\vec{h}_u \parallel \vec{m}_u \parallel \vec{n}_u]) @ \mathbf{W}_3^\top && \text{(aggregate \& project)}\end{aligned}$$

Stacking K such layers inside one transformer block gives a K -hop local context before attention, following the packed edge orientation used by the implementation.

3.5 Attention Sublayer (`models/attention.py`)

Multi-head scaled dot-product attention is applied to *all* tokens (nodes + CLS):

$$a_{ij}^\ell = \frac{(\vec{h}_i @ \mathbf{W}_Q^{\ell\top})(\vec{h}_j^\top @ \mathbf{W}_K^{\ell\top})}{\sqrt{d_h}}, \quad \tilde{a}_{ij}^\ell = a_{ij}^\ell + \log A_{ij}^*, \quad \vec{h}'_i = \sum_j \text{softmax}_j(\tilde{a}_i^\ell) \vec{h}_j @ \mathbf{W}_V^{\ell\top},$$

where $A_{ij}^* \in [0, 1]$ is the precomputed reachability entry. Because $\log 0 = -\infty$, unreachable pairs are effectively masked out. The CLS token is bidirectionally connected to every node in its graph (set explicitly in `pack_graph_batch`), so it can attend to—and be attended by—all nodes. Cross-graph attention is prevented by the block-diagonal structure of $\mathbf{A}^*$. The multi-head outputs are concatenated and passed through a final linear output projection.

Optional distance-weighted attention. The config parameter `attn_distance_factors` (a list of per-hop scale factors) allows replacing the binary reachability mask with a soft, distance-decayed version: nodes k hops away contribute with factor f_k . In our experiments this is set to `None`, giving binary transitive-closure reachability with no distance decay.

3.6 MLP Sublayer (`models/mlp.py`)

A position-wise two-layer feedforward network with GELU activation is applied after attention. A key implementation detail: the MLP module maintains **two independent sets of weights**—`layers` for regular nodes and `layers_cls` for the CLS token. This lets the model learn different transformations for the graph-summary token versus per-node representations, without coupling their learning dynamics.

3.7 Output Head

After L transformer blocks, the CLS embedding $\vec{h}_{\text{cls}} \in \mathbb{R}^d$ is extracted and passed through the output head:

- *Classification* (NCI, OGBG MolPCBA): $\hat{y} = \mathbf{W}_{\text{out}} \vec{h}_{\text{cls}}$, $\mathbf{W}_{\text{out}} \in \mathbb{R}^{d \times C}$.
- *Sequence prediction* (OGBG Code2): $\hat{y}_t = \mathbf{W}_{\text{out}}^{(t)} \vec{h}_{\text{cls}}$ for $t = 1, \dots, T_{\text{max}}$, using T_{max} independent linear heads.

4 Training Objective and Optimization (`models/train.py`)

Loss. For graph classification with batch size B :

$$\mathcal{L}(\theta) = \frac{1}{B} \sum_{g=1}^B \text{CE}(f_{\theta}(G_g), y_g).$$

For Code2, cross-entropy is averaged over all sequence positions:

$$\mathcal{L}(\theta) = \frac{1}{B \cdot T} \sum_{g=1}^B \sum_{t=1}^T \text{CE}(\hat{y}_t^{(g)}, y_t^{(g)}).$$

Optimizer and scheduler. All experiments use Adam with $(\beta_1, \beta_2) = (0.9, 0.999)$. A cosine-annealing LR schedule (`CosineAnnealingLR`) with $T_{\text{max}} = \text{\#epochs}$ is applied unless disabled. During training, the best model state by validation metric is kept in memory and restored before reporting test results; `main.py` then saves checkpoints to disk.

Multi-run protocol. `run_graph_transformer_experiments` trains R independent runs with different seeds. Final results are reported as mean $\pm$ std of test metrics across runs. Exact reproducibility can still depend on hardware backend and nondeterministic PyTorch/CUDA kernels.

4.1 FLAG Adversarial Training (`trainers/flag_trainer.py`)

The repository contains a FLAG [13] trainer, which augments training by injecting adversarial perturbations δ into node embeddings, computed via m projected gradient steps:

$$\delta^{t+1} = \Pi_{\|\cdot\|_{\infty} \leq \epsilon}(\delta^t + \alpha \text{sign}(\nabla_{\delta} \mathcal{L}(\theta, \delta^t))),$$

with step size α (default 8×10^{-3}) and $m = 3$ steps. However, this trainer is not used by the current `models/train.py` `GraphTransLWG` training loop, and it expects a model interface with an explicit perturbation input. We therefore treat it as auxiliary repository code rather than part of the main reported experiments.

5 Implementation Summary

- `models/full_model.py`: `GraphTransConfig` (hyperparameter dataclass), `GraphTransModel` (full pipeline), `ASTNodeEncoder` (code2 input), `ModelTrainConfig` (training config).
- `models/gnn.py`: `GNNLayer` (edge-aware single layer), `GNN` (stack of GNN layers).
- `models/attention.py`: `AttentionLayer` (reachability-masked attention with optional distance factors).

- `models/mlp.py`: MLP (position-wise FFN with separate CLS weights).
- `models/transformer.py`: `TransformerLayer` (GNN + Attn + MLP with pre-norm residuals), `Transformer` (stack).
- `models/batch_utils.py`: `PackedGraphBatch`, `pack_graph_batch`, `build_attn_factors` (precomputation of $\mathbf{A}^*$, the CLS mask and the batch edge embedding for reuse).
- `models/train.py`: `train_graph_transformer`, `run_graph_transformer_experiments` (multi-run training loop with tracking of mean and standard deviation of a metric, and the loss).
- `data_utils/config_objects.py`: per-dataset `GraphTransConfig` and `ModelTrainConfig` instances.
- `data_utils/code2_train_config.py`: code2-specific config factory (`ASTNodeEncoder` setup, vocab building, F1 metric).
- `code2_gnn_initialization_ablation/configs.py`: config factories for the Code2 dataset – GNN-only/frozen-GNN/fine-tuned-GNN ablation – kept as auxiliary code.
- `nci_readout_ablation/model.py`, `nci_readout_ablation/main.py`: ablation code for NCI1/NCI109 variants.
- `trainers/flag_trainer.py`: auxiliary FLAG adversarial perturbation trainer, not used in the main `models/train.py` loop.

6 Experiments

6.1 Biological Benchmarks (NCI1 & NCI109)

Dataset. NCI1 and NCI109 each contain $\approx 4,000$ graphs (≈ 30 nodes on average) representing biochemical compounds. The task is binary classification: predicting anti-lung-cancer activity. We follow prior work and split 8:1:1 (train/val/test) with random seeds.

Our experimental setup. *50 epochs*, batch size *128*, Adam ($\beta_1=0.9$, $\beta_2=0.999$), $\text{lr} = 10^{-4}$, weight decay $= 10^{-4}$, cosine annealing schedule, *5 independent runs*. Architecture: 4 transformer layers; per block: 3 GNN layers, 8-head attention (head dim 16), 2-layer MLP (hidden 512); embed dim 128; dropout 0.1.

6.2 Computer Programming Benchmark (OGBG Code2)

Dataset. `ogbg-code2` contains 45,741 graphs—Abstract Syntax Trees of Python method bodies—with 125 nodes on average. The task is sequence prediction: given the AST, predict the sub-tokens of the method name. Performance is measured by token-level F1 score using the OGB evaluator. We use the standard fixed OGB splits.

Our experimental setup. *30 epochs*, batch size *16*, Adam, $\text{lr} = 10^{-4}$, weight decay $= 0$, cosine annealing schedule, *5 independent runs*. Architecture: 4 transformer layers; per block: 4 GNN layers, 6-head attention (head dim 50), 2-layer MLP (hidden 600); embed dim 300; dropout 0.3. Input via `ASTNodeEncoder` (node-type + attribute + depth embeddings summed). Output: $T_{\max} = 5$ independent linear heads over vocabulary.

Table 1: NCI1 and NCI109 test accuracy (%). Paper baselines reproduced from [1]

Model	GNN Type	GNN layers	NCI1 (%)	NCI109 (%)
Set2Set [9]	GCN	3	68.6 ± 1.9	69.8 ± 1.2
SortPool [10]	GCN	3	73.8 ± 1.0	74.0 ± 1.2
SAGPool _h [8]	GCN	3	67.5 ± 1.1	67.9 ± 1.4
SAGPool _g [8]	GCN	3	74.2 ± 1.2	74.1 ± 0.8
Errica et al. [11]	GIN	8	80.0 ± 1.4	—
Alon & Yahav [12]	GIN	8	81.5 ± 1.2	—
Transformer [2]	—	—	68.5 ± 2.6	70.1 ± 2.3
GraphTrans (small) [1]	GCN	3	81.3 ± 1.9	79.2 ± 2.2
GraphTrans (large) [1]	GIN	4	82.6 ± 1.2	82.3 ± 2.6
GraphTransLWG (ours)	custom	3 [†]	78.59 ± 1.79	76.47 ± 1.42

[†]GNN layers per transformer block (3 layers $\times$ 4 blocks = 12 total GNN applications); baselines report total backbone GNN layers.

Table 2: OGB Code2 **validation** F1 score. Baselines from OGB leaderboard [7] (originally reported on the test set; our result is validation F1 only, so direct comparison should be interpreted with caution).

Model	Mean F1 $\pm$ std dev
GIN [3]	0.1376 ± 0.0016
GCN [4]	0.1399 ± 0.0017
GIN-Virtual [3]	0.1439 ± 0.0026
GCN-Virtual [4]	0.1461 ± 0.0013
PNA [5]	0.1453 ± 0.0025
DAGNN [6]	0.1607 ± 0.0040
Transformer [2]	0.1546 ± 0.0018
GraphTrans (GCN) [1]	0.1599 ± 0.0009
GraphTrans (PNA) [1]	0.1622 ± 0.0025
GraphTrans (GCN-Virtual) [1]	0.1661 ± 0.0012
GraphTransLWG (ours)	0.1735 ± 0.03

6.3 Ablation: Effectiveness of [CLS] Embedding on NCI1

Setup. We perform an ablation study on different Transformer readout strategies to analyze how graph-level representations should be extracted from the output token sequence. The compared readout strategies are: (1) **mean**: average all node (non-CLS) output embeddings to obtain one graph embedding; (2) **last**: use the last output embedding as the graph embedding; (3) **[CLS]**: append a learnable [CLS] token and use its final embedding as the graph representation; and (4) **[CLS], cat**: concatenate the final [CLS] embedding with the mean of the final node embeddings before prediction.

This ablation evaluates whether a dedicated graph-summary token can aggregate graph-level information more effectively than standard pooling operations.

GraphTransLWG setup. The experiments are conducted on the NCI1 dataset using our GRAPHTRANSLWG architecture. Each configuration was evaluated across multiple random

seeds, and we report the mean and standard deviation of the test accuracy.

Table 3: Ablation of readout strategies on NCI1 using GRAPHTRANSLWG.

Readout	Test Accuracy (%)
[CLS]	79.72 ± 3.22
mean	81.51 ± 1.58
last	81.59 ± 2.85
[CLS], cat	81.67 ± 2.11

The quantitative results show that the vanilla [CLS] readout performs worse than the alternative aggregation strategies, achieving only 79.72% test accuracy. In contrast, both mean pooling and last-token pooling achieve comparable performance above 81.5%. The best result is obtained using the concatenated [CLS] variant, which achieves 81.67% test accuracy.

These observations suggest that while a standalone [CLS] token may not sufficiently capture graph-level structure in GRAPHTRANSLWG, augmenting it with a pooled node-level summary significantly improves representation quality. Overall, the [CLS], cat strategy provides the most effective graph-level aggregation among the evaluated methods.

7 Conclusion

We implemented GRAPHTRANSLWG, a GraphTrans-style architecture [1] with several implementation-level contributions: (i) a custom edge-aware GNN layer that incorporates pairwise edge embeddings alongside neighbour aggregation; (ii) transitive-closure-based attention masking precomputed once per batch and reused across all L transformer blocks; (iii) separate MLP weights for the [CLS] token; and (iv) a pre-norm residual structure throughout. Experiments on NCI1, NCI109, and OGB Code2 compare our implementation against standard baselines, while the readout ablation (Table 3) evaluates how different graph-summary strategies affect GRAPHTRANSLWG.

References

- [1] Z. Wu, P. Jain, M. A. Wright, A. Mirhoseini, J. E. Gonzalez, I. Stoica. Representing Long-Range Context for Graph Neural Networks with Global Attention. *NeurIPS 2021*.
- [2] A. Vaswani et al. Attention is All You Need. *NeurIPS 2017*.
- [3] K. Xu, W. Hu, J. Leskovec, S. Jegelka. How Powerful are Graph Neural Networks? *ICLR 2019*.
- [4] T. N. Kipf, M. Welling. Semi-Supervised Classification with Graph Convolutional Networks. *ICLR 2017*.
- [5] G. Corso et al. Principal Neighbourhood Aggregation for Graph Nets. *NeurIPS 2020*.
- [6] V. Thost, J. Chen. Directed Acyclic Graph Neural Networks. *ICLR 2021*.
- [7] W. Hu et al. Open Graph Benchmark: Datasets for Machine Learning on Graphs. *NeurIPS 2020*.
- [8] J. Lee, I. Lee, J. Kang. Self-Attention Graph Pooling. *ICML 2019*.
- [9] O. Vinyals, S. Bengio, M. Kudlur. Order Matters: Sequence to Sequence for Sets. *ICLR 2016*.

- [10] M. Zhang, Z. Cui, M. Neumann, Y. Chen. An End-to-End Deep Learning Architecture for Graph Classification. *AAAI 2018*.
- [11] F. Errica, M. Podda, D. Bacciu, A. Micheli. A Fair Comparison of Graph Neural Networks for Graph Classification. *ICLR 2020*.
- [12] U. Alon, E. Yahav. On the Bottleneck of Graph Neural Networks and its Practical Implications. *ICLR 2021*.
- [13] K. Kong, G. Li, M. Ding, Z. Wu, C. Zhu, B. Ghanem, G. Taylor, T. Goldstein. FLAG: Adversarial Data Augmentation for Graph Neural Networks. *arXiv 2020*.